

UNIVERSIDAD EAFIT
ESCUELA DE CIENCIAS APLICADAS E INGENIERÍA
ARQUITECTURA DE SOFTWARE 2026

Tutorial 03: Introducción a APIs con DRF

De Monolito HTML a Backend Headless

Nicolás Rodríguez

Profesor: Nicolás Ramírez Vélez

30 de septiembre de 2026

1. Objetivo cumplido en una línea

Dos puertas, una sola habitación: la vista HTML y el nuevo endpoint `POST /api/v1/comprar/` llaman **al mismo** `CompraService`. No reescribí ni una línea de lógica de negocio; la API es solo un cliente más.

2. Evidencia: POST desde la interfaz web de DRF

Autenticado como `nicolas`, envié el payload `{"libro_id": 2, "direccion_envio": "Cra 49 #7 Sur-50, Medellín"}`. El serializer validó la entrada, la Factory entregó el procesador del Banco Nacional (modo por defecto) y el Service Layer respondió **HTTP 201 Created**:

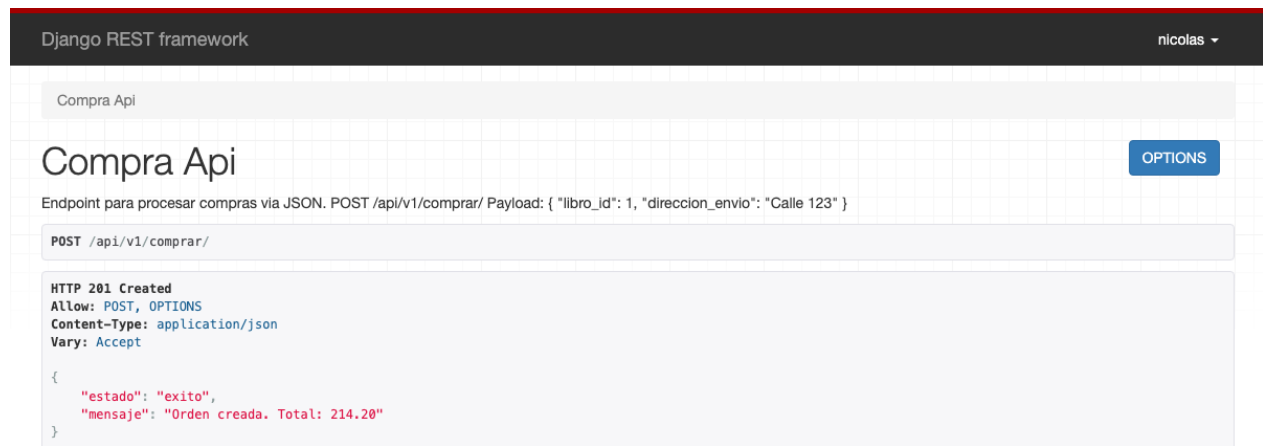


Figura 1: POST a `/api/v1/comprar/` desde la interfaz navegable de DRF: 201 Created, total \$214.20 (\$180 + IVA).

Para cubrir también el caso de un cliente externo (tipo Postman), hice una segunda compra con autenticación Basic:

```
curl -i -u nicolas:**** -H "Content-Type: application/json" -d '{"libro_id": 4, ...}'  
/api/v1/comprar/
```

```
HTTP/1.1 201 Created  
Date: Wed, 30 Sep 2026 10:46:47 GMT
```

```

Server: WSGIServer/0.2 CPython/3.13.7
Content-Type: application/json
Vary: Accept, Cookie
Allow: POST, OPTIONS
X-Frame-Options: DENY
Content-Length: 58
X-Content-Type-Options: nosniff
Referrer-Policy: same-origin
Cross-Origin-Opener-Policy: same-origin

{"estado":"exito","mensaje":"Orden creada. Total: 249.90"}

```

3. Evidencia: la vista HTML refleja el cambio

El inventario HTML (servido por la vista clásica de Django) antes y después de las dos compras por API:

Inventario de la Tienda

ID	Libro	Precio	Stock	
1	Cien años de soledad	\$150,00	10	Compra rápida
2	Clean Architecture	\$180,00	10	Compra rápida
3	Patrones de Diseño (GoF)	\$165,00	10	Compra rápida
4	Domain-Driven Design	\$210,00	10	Compra rápida

Órdenes registradas: 0

Figura 2: Antes: todos los libros con stock 10 y 0 órdenes.

Inventario de la Tienda

ID	Libro	Precio	Stock	
1	Cien años de soledad	\$150,00	10	Compra rápida
2	Clean Architecture	\$180,00	9	Compra rápida
3	Patrones de Diseño (GoF)	\$165,00	10	Compra rápida
4	Domain-Driven Design	\$210,00	9	Compra rápida

Órdenes registradas: 2

Figura 3: Después: Clean Architecture (#2) y Domain-Driven Design (#4) bajan a 9; 2 órdenes registradas.

4. Archivo de log de pagos

Generado por BancoNacionalProcesador (la implementación que entrega la Factory del Tutorial 02). Se adjunta como archivo aparte:

pagos_locales_NicolasRodriguez.log

```
[2026-09-30 05:46:38.965663] Transaccion exitosa por: $214.20
[2026-09-30 05:46:47.937645] Transaccion exitosa por: $249.90
[2026-09-30 05:51:53.292017] Transaccion exitosa por: $214.20
[2026-09-30 05:57:29.097935] Transaccion exitosa por: $196.35
```

5. Código implementado

tienda_app/api/serializers.py (Adapter / DTO)

```
from rest_framework import serializers
from tienda_app.models import Libro, Orden

class LibroSerializer(serializers.ModelSerializer):
    class Meta:
        model = Libro
        fields = ['id', 'titulo', 'precio', 'stock_actual']
        # Nota: 'stock_actual' es una propiedad del modelo (lee el Inventario)

class OrdenInputSerializer(serializers.Serializer):
    """
    Serializer para VALIDAR la entrada de datos, no necesariamente ligado a un modelo.
    Actua como un DTO (Data Transfer Object).
    """
    libro_id = serializers.IntegerField()
    direccion_envio = serializers.CharField(max_length=200)
    # Validaciones extras aqui si fueran necesarias
```

tienda_app/api/views.py (APIView → Service Layer)

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from rest_framework.generics import ListAPIView

from .serializers import LibroSerializer, OrdenInputSerializer
from tienda_app.models import Libro
from tienda_app.services import CompraService # <--- REUTILIZACION
from tienda_app.infra.factories import PaymentFactory

class CompraAPIView(APIView):
    """
    Endpoint para procesar compras via JSON.
    POST /api/v1/comprar/
    Payload: { "libro_id": 1, "direccion_envio": "Calle 123" }
    """
    def post(self, request):
        # 1. Validacion de datos de entrada (Adapter)
        serializer = OrdenInputSerializer(data=request.data)
        if not serializer.is_valid():
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

```

datos = serializer.validated_data
try:
    # 2. Inyeccion de Dependencias (Factory)
    gateway = PaymentFactory.get_processor()

    # 3. Ejecucion de Logica de Negocio (Service Layer)
    servicio = CompraService(procesador_pago=gateway)

    # Noten como el servicio NO cambia, solo cambia quien lo llama.
    resultado = servicio.ejecutar_compra(
        libro_id=datos['libro_id'],
        direccion=datos['direccion_envio'],
        usuario=request.user if request.user.is_authenticated else None,
    )
    return Response({
        "estado": "exito",
        "mensaje": f"Orden creada. Total: {resultado}"
    }, status=status.HTTP_201_CREATED)

except ValueError as e:
    # Errores de negocio (ej: Sin stock, datos insuficientes)
    return Response({"error": str(e)}, status=status.HTTP_409_CONFLICT)
except Exception:
    # Errores inesperados
    return Response({"error": "Error interno"}, status=status.
HTTP_500_INTERNAL_SERVER_ERROR)

class LibroListAPIView(ListAPIView):
    """
    GET /api/v1/productos/ - Catalogo servido por el monolito Django (v1).
    Lectura pura sobre la BD: aqui basta una vista generica (sin Service Layer).
    """
    queryset = Libro.objects.select_related('inventario').order_by('id')
    serializer_class = LibroSerializer

```

tienda_app/urls.py

```
path('api/v1/comprar/', CompraAPIView.as_view(), name='api_comprar'),
```

Decisiones puntuales: `stock_actual` es una propiedad del modelo `Libro` que lee su `Inventario`; habilité autenticación por **sesión** (interfaz de DRF) y **Basic** (Postman/curl); y si la petición llega anónima, el `OrdenBuilder` la rechaza y la API responde **409** en lugar de romperse. Tres pruebas nuevas cubren 201, 409 (sin stock) y 400 (payload inválido); la suite completa (10 tests) pasa.

Con Postman: POST `http://localhost:8000/api/v1/comprar/` · Authorization: Basic Auth (nicolas / tienda2026) · Body → raw → JSON: `{"libro_id": 1, "direccion_envio": "Calle 123"}`.